

# Reinforcement Learning – Projects Examples

## Project 1: Taxi environment with Q-Learning

Let's design a simulation of a self-driving cab. The major goal is to demonstrate, in a simplified environment, how you can use RL techniques to develop an efficient and safe approach for tackling this problem. The Smartcab's job is to pick up the passenger at one location and drop them off in another. Here are a few things that we'd love our Smartcab to take care of:

- Drop off the passenger to the right location.
- Save passenger's time by taking minimum time possible to drop off
- Take care of passenger's safety and traffic rules

There are different aspects that need to be considered here while modeling an RL solution to this problem: rewards, states, and actions.

This workbook provides a step-to-step guide to implement the environment, learn a policy using tabular Q-learning, and visualize the learned behaviour in animations.

### Problem setting

In the Taxi environment, a taxi should pick up passengers and drop them at their destination on a small parking lot, driving the shortest path possible. How hard can it be to learn such a task with RL? Let's find out.

Before diving into the implementation, it is good practice to first formulate the setting as a [Markov Decision Process](#) (MDP), getting a grasp on the mathematical structure of the problem at hand.

### State space

An MDP states contains the information needed to (i) determine actions, (ii) compute the action reward and (iii) compute the transition to the next state.

The most obvious state property is the location of the taxi; the 5\*5 grid gives us 25 options. Furthermore, a passenger waiting for pickup can be waiting at 1 or 4 points

(labeled Y, R, G, B) or they can be in the taxi, giving us (4+1) options. Finally, the destination can also be Y, R, G or B. We denote the state by the following vector:

State = [x\_pos\_taxi, y\_pos\_taxi, pos\_passenger, dest\_passenger]

Although a very simple problem setting, we thus already deal with  $5*5*(4+1)*4=500$  states! Scale the problem to thousands of locations, and it is easy to see how quickly the size of MDPs explodes (the so-called curse of dimensionality).

*The number of **reachable** states is slightly smaller than 500, e.g., a passenger will never have the same pickup point and destination. Due to modelling complexities, we typically focus on the full state space, representing the worst-case scenario.*

## Action space

The state space may be bigger than you'd expect at first glance, but the action space contains only six decisions:

- Down
- Up
- Left
- Right
- Drop passenger
- Pick up passenger

In case the action is not allowed (e.g., trying to drive through a wall, attempting to pick up a customer when there is none at the present location), the agent will remain still.

The action\_mask (found in env.action\_mask(state)) informs us which actions are feasible, but this feasibility check is not automatically enforced.

## Reward function

Three types of reward can be distinguished in this environment:

- **Move: -1.** Every move comes with a small penalty, to encourage following the shortest path from origin to destination.
- **Failed drop-off: -10.** A passenger will naturally be unhappy when dropped at the wrong location, so a large penalty is appropriate.
- **Successful drop-off: 20.** To encourage the desired behaviour, a large bonus is incurred.

*Designing rewards in a way that encourages desired behaviour is known as reward shaping. For instance, we might simply end the game without a reward upon successful drop-off (thus ending the stream of costs), but a clear reward signal often eases learning considerably.*

## Transition function

Unlike most realistic environments, the taxi environment exhibits no randomness. This feature considerably simplifies learning, as the **next state is dictated solely by our action**. In case an infeasible action is selected — such as trying to drive through a wall or picking up a non-existent passenger — the agent simply remains in place.

In an abstract sense, we could denote the transition function as follows:

$$new\_state = f(state, action)$$

An explicit definition would consider all grid moves, (in)feasible drop-offs and (in)feasible pickups, which is not necessarily hard but requires some focus. The game ends upon successfully dropping off a passenger at their destination, denoting the terminating state.

## Initialization

Having mathematically defined the problem (although not in excruciating detail), we will move towards implementation. First, we install the necessary libraries and subsequently import them. Obviously, we need to install the gym environment. Other than that, we just require some visual stuff and your typical data science libraries.

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
import random
from IPython.display import clear_output
from time import sleep
from matplotlib import animation
```

Assuming all imports went correctly, it is time for a sanity check. Let's create and render a Taxi-v3 environment.

```
"""Initialize and validate the environment"""
env = gym.make("Taxi-v3", render_mode="rgb_array").env
state, _ = env.reset()

# Print dimensions of state and action space
print("State space: {}".format(env.observation_space))
print("Action space: {}".format(env.action_space))

# Sample random action
action = env.action_space.sample(env.action_mask(state))
next_state, reward, done, _, _ = env.step(action)

# Print output
print("State: {}".format(state))
print("Action: {}".format(action))
print("Action mask: {}".format(env.action_mask(state)))
print("Reward: {}".format(reward))

# Render and plot an environment frame
frame = env.render()
plt.imshow(frame)
plt.axis("off")
plt.show()
```

We should see an action between 0 and 5, a state between 0 and 499, and an action mask to block infeasible actions. Most importantly, we should have rendered a still frame to affirm we are truly up and running!

```
State space: Discrete(500)
Action space: Discrete(6)
State: 424
Action: 1
Action mask: [0 1 1 0 0 0]
Reward: -1
```



Finally, let's set up two functions: one for the console output and animation, one for storing GIFs. Note that the animation is simply a repeated plot of the RGB frame. It is also very time-consuming, and therefore not recommended to run during training.

```
def run_animation(experience_buffer):
    """Function to run animation"""
    time_lag = 0.05 # Delay (in s) between frames
    for experience in experience_buffer:
        # Plot frame
        clear_output(wait=True)
        plt.imshow(experience['frame'])
        plt.axis('off')
        plt.show()

        # Print console output
        print(f"Episode: {experience['episode']}/{experience_buffer[-1]['episode']}")
        print(f"Epoch: {experience['epoch']}/{experience_buffer[-1]['epoch']}")
        print(f"State: {experience['state']}")
        print(f"Action: {experience['action']}")
        print(f"Reward: {experience['reward']}")
        # Pause animation
        sleep(time_lag)
```

```

def store_episode_as_gif(experience_buffer, path='./', filename='animation.gif'):
    """Store episode as gif animation"""
    fps = 5 # Set framew per seconds
    dpi = 300 # Set dots per inch
    interval = 50 # Interval between frames (in ms)

    # Retrieve frames from experience buffer
    frames = []
    for experience in experience_buffer:
        frames.append(experience['frame'])

    # Fix frame size
    plt.figure(figsize=(frames[0].shape[1] / dpi, frames[0].shape[0] / dpi), dpi=dpi)
    patch = plt.imshow(frames[0])
    plt.axis('off')

    # Generate animation
    def animate(i):
        patch.set_data(frames[i])

    anim = animation.FuncAnimation(plt.gcf(), animate, frames=len(frames), interval=interval)

    # Save output as gif
    anim.save(path + filename, writer='imagemagick', fps=fps)

```

## Testing a random agent

Having asserted that the environment works as expected, it is time to let a random agent run wild. As you guessed, this agent takes random action at every moment in time. This is essentially how the learning agent sets out, having **no reliable Q-values** to work with.

You will see the agent trying to drive through walls, pick up passengers at deserted spots, or drop off passenger at the wrong location. It might take thousands of actions before — strictly by accident — dropping the passenger at the right location.

```

"""Simulation with random agent"""
epoch = 0
num_failed_dropoffs = 0
experience_buffer = []
cum_reward = 0

done = False

state, _ = env.reset()

while not done:
    # Sample random action
    "Action selection without action mask"
    action = env.action_space.sample()

    "Action selection with action mask"
    #action = env.action_space.sample(env.action_mask(state))

    state, reward, done, _, _ = env.step(action)
    cum_reward += reward

    # Store experience in dictionary
    experience_buffer.append({
        "frame": env.render(),
        "episode": 1,
        "epoch": epoch,
        "state": state,
        "action": action,
        "reward": cum_reward,
    })

    if reward == -10:
        num_failed_dropoffs += 1

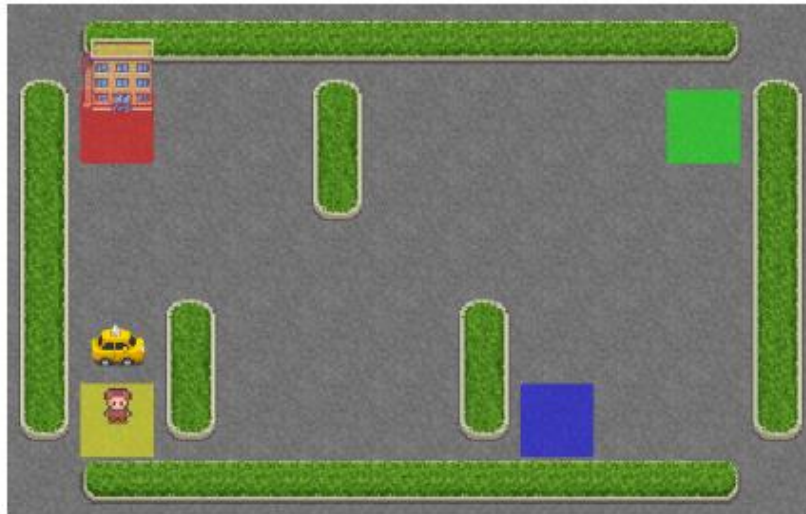
    epoch += 1

# Run animation and print console output
run_animation(experience_buffer)

print("# epochs: {}".format(epoch))
print("# failed drop-offs: {}".format(num_failed_dropoffs))

```

As you can imagine, the random agent frustrates the passenger to no end:



Animation of an untrained agent. The taxi selects random actions at each time step, including infeasible ones, until eventually dropping of the passenger at the correct stop by accident

Why sit through this lengthy animation? Well, it really gives an impression how an untrained RL agent behaves, and how long it takes to get a meaningful reward signal.

*Note: by applying the **action mask** — a Boolean vector that essentially assigns a probability of 0 to selecting infeasible actions — we restrict ourselves to only sensible actions, typically substantially shortening the episode's length.*

## Training the agent

Time to learn something useful.

We will deploy a [basic Q-learning algorithm](#) with  $\epsilon$ -greedy exploration, selecting random actions with probability  $\epsilon$  and use Q-values to select actions for the remainder of the time.

Q-values are updates using the following equation after making an observation. Note that with 500 states and 6 actions, we must fill a Q-table of size  $500 \times 6 = 3000$ , with each state-action pair requiring multiple observations to learn its value with any degree of accuracy.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left( r_{t+1} + \gamma \max_{a' \in \mathcal{A}(s_t)} Q(s_{t+1}, a') - Q(s_t, a_t) \right)$$

In Q-learning, we update values of state-action pairs after each observation, using off-policy learning.

The code looks as follows. You can tinker with the hyper-parameters; in my experience this problem is fairly insensitive to them.

```

"""Training the agent"""
q_table = np.zeros([env.observation_space.n, env.action_space.n])

# Hyperparameters
alpha = 0.1 # Learning rate
gamma = 1.0 # Discount rate
epsilon = 0.1 # Exploration rate
num_episodes = 10000 # Number of episodes

# Output for plots
cum_rewards = np.zeros([num_episodes])
total_epochs = np.zeros([num_episodes])

for episode in range(1, num_episodes+1):
    # Reset environment
    state, info = env.reset()
    epoch = 0
    num_failed_dropoffs = 0
    done = False
    cum_reward = 0

    while not done:

        if random.uniform(0, 1) < epsilon:
            "Basic exploration [~0.47m]"
            action = env.action_space.sample() # Sample random action (exploration)
            "Exploration with action mask [~1.52m]"
            # action = env.action_space.sample(env.action_mask(state)) "Exploration with action mask"
        else:
            "Exploitation with action mask [~1m52s]"
            # action_mask = np.where(info["action_mask"]==1,0,1) # invert
            # masked_q_values = np.ma.array(q_table[state], mask=action_mask, dtype=np.float32)
            # action = np.ma.argmax(masked_q_values, axis=0)

            "Exploitation with random tie breaker [~1m19s]"
            # action = np.random.choice(np.flatnonzero(q_table[state] == q_table[state].max()))

            "Basic exploitation [~47s]"
            action = np.argmax(q_table[state]) # Select best known action (exploitation)

    next_state, reward, done, _, info = env.step(action)

    cum_reward += reward

    old_q_value = q_table[state, action]
    next_max = np.max(q_table[next_state])

    new_q_value = (1 - alpha) * old_q_value + alpha * (reward + gamma * next_max)

    q_table[state, action] = new_q_value

```

```

if reward == -10:
    num_failed_dropoffs += 1

state = next_state
epoch += 1

total_epochs[episode-1] = epoch
cum_rewards[episode-1] = cum_reward

if episode % 100 == 0:
    clear_output(wait=True)
    print(f"Episode #: {episode}")

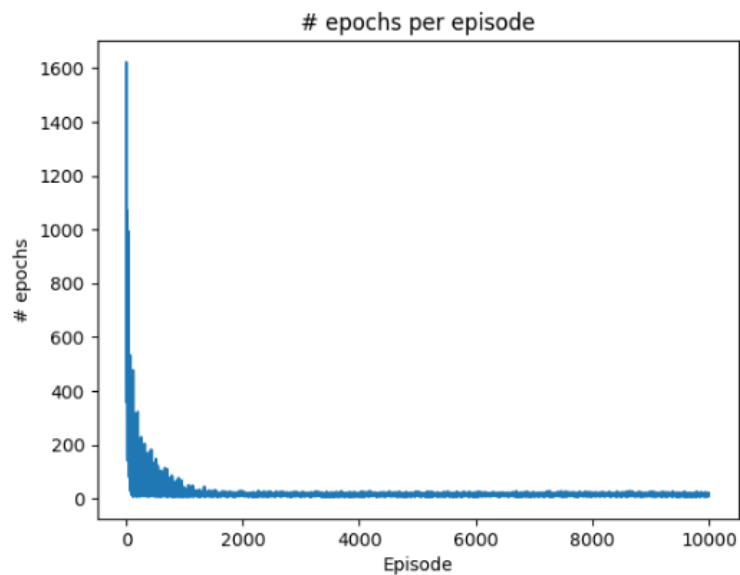
print("\n")
print("===Training completed.===\n")

# Plot reward convergence
plt.title("Cumulative reward per episode")
plt.xlabel("Episode")
plt.ylabel("Cumulative reward")
plt.plot(cum_rewards)
plt.show()

# Plot epoch convergence
plt.title("# epochs per episode")
plt.xlabel("Episode")
plt.ylabel("# epochs")
plt.plot(total_epochs)
plt.show()

```

Convergence of training is visualized below. After 2000 episodes, it seems we have learned a pretty good and consistent policy.



Initially, the agent requires many steps to successfully complete an episode. Ultimately, positive rewards are recognized, and the agent starts taking increasingly efficient actions, culminating in shortest path solutions.

*Note: the code ignores the action mask and selects the first Q-value in case of a tie. Although alternatives for both issues are provided, they substantially increase the computational effort per episode, while providing no clear performance benefit in this setting.*

## Testing the policy

Let's see what we learned. Depending on the state we are in, we look up the corresponding Q-values in the Q-table (i.e., six values for each state, corresponding to the actions) and select the action with the highest associated Q-value.

Effectively, the Q-value captures the expected cumulative reward associated with the action. Note that we no longer select random actions with probability  $\epsilon$  — this mechanism was required strictly for learning.

```
"""Test policy performance after training"""

num_epochs = 0
total_failed_deliveries = 0
num_episodes = 1
experience_buffer = []
store_gif = True

for episode in range(1, num_episodes+1):
    # Initialize experience buffer

    my_env = env.reset()
    state = my_env[0]
    epoch = 1
    num_failed_deliveries = 0
    cum_reward = 0
    done = False

    while not done:
        action = np.argmax(q_table[state])
        state, reward, done, _, _ = env.step(action)
        cum_reward += reward

        if reward == -10:
            num_failed_deliveries += 1

        # Store rendered frame in animation dictionary
        experience_buffer.append({
            'frame': env.render(),
            'episode': episode,
            'epoch': epoch,
            'state': state,
            'action': action,
            'reward': cum_reward
        })

        epoch += 1

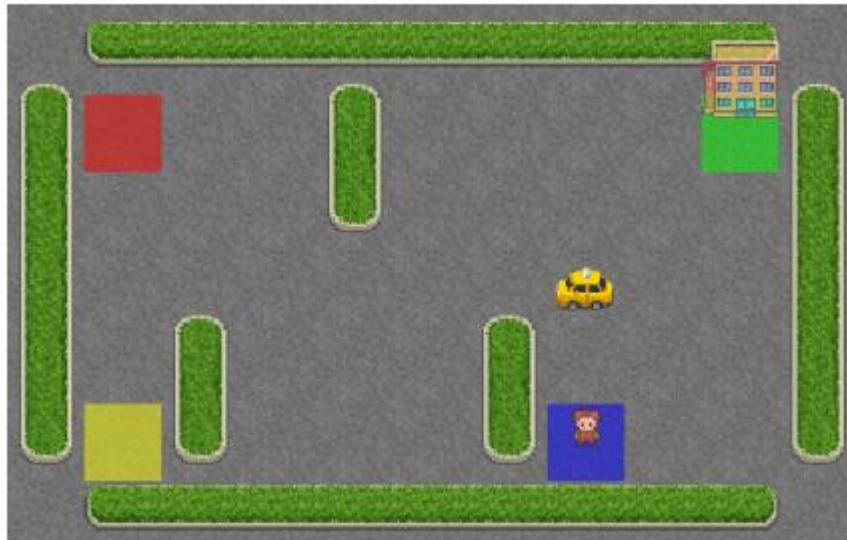
    total_failed_deliveries += num_failed_deliveries
    num_epochs += epoch

    if store_gif:
        store_episode_as_gif(experience_buffer)

# Run animation and print output
run_animation(experience_buffer)

# Print final results
print("\n")
print(f"Test results after {num_episodes} episodes:")
print(f"Mean # epochs per episode: {num_epochs / num_episodes}")
print(f"Mean # failed drop-offs per episode: {total_failed_deliveries / num_episodes}")
```

If you performed sufficiently many iterations, you should see the taxi always drive directly to the passenger, take the shortest path to the destination, and successfully drop off the passenger. Indeed, we've come a long way since driving around at random.



Animation of a successfully learned policy. The taxi will follow the shortest path both to pick up and drop off the passenger, maximizing the reward.

Although it requires many observations to learn this policy in a seemingly trivial environment, you might appreciate the result more if you realize the algorithm learned all of this without knowing anything at all about the environment it operates in!

## Wrapping up

The Taxi environment is a nice one to get started with Reinforcement Learning. The problem setting is simple and intuitive yet could easily be extended towards something more realistic (think realistic urban networks, managing taxi fleets, uncertain travel times). It is computationally well-manageable, but also illustrates the computational effort required even to learn policies for trivial problem settings.

If you are new to Reinforcement Learning, playing around with the OpenAI Gym/Gymnasium environments is highly recommended before moving to designing and solving your own problems.

## Future work

If you'd like to continue with this project to make it better, here's a few things you can add:

- Turn this code into a module of functions that can use multiple environments.
- Tune alpha, gamma, and/or epsilon using a decay over episodes.
- Implement a grid search to discover the best hyperparameters.

---

## Project 2: Self-driving cab with SARSA (State-Action-Reward-State-Action)

Now let's explore how another classical RL algorithm, SARSA, usually considered an alternative to Q-learning works, and we will implement it in the same environment used earlier (i.e., self-driving taxi).

## Introduction

The beauty of Machine Learning is that there is no shortage of approaches for tackling complex tasks. For example, **Reinforcement Learning (RL)** practitioners have developed multiple algorithms capable of teaching **intelligent agents** to navigate their environments and perform actions. In this part, I dive deeper into an RL algorithm called **SARSA** (State-Action-Reward-State-Action), a close sibling of Q-Learning.

## SARSA's place in the universe of Machine Learning (ML) algorithms

Similarities exist between Machine Learning algorithms that enable us to categorise them based on architecture and use cases. Hence, I have created a visualisation which helps us see the whole ML universe. The **chart is interactive**, so please click  on different categories to explore and reveal more.

You will find **SARSA** under the **Reinforcement Learning** branch of ML algorithms within the group of **Value-based methods** (more on this in the next section).

## How does SARSA work?

### Q-table

SARSA is a **value-based method** similar to Q-learning. Hence, it uses a **Q-table** to store values for each **state-action pair**. With value-based strategies, we train the agent **indirectly** by teaching it to identify which states (or state-action pairs) are more valuable.

Typically, we start with all values in a Q-table initialised to 0, and we use training to optimise the Q-table. Then our agent can use information stored in a Q-table to choose the best action at each state (i.e., the action with the highest value for each state is the one selected by the agent). Here is an example of an empty Q-table:

|             |         | Action Space |            |     |              |            |
|-------------|---------|--------------|------------|-----|--------------|------------|
| State Space | Q-table | Action (0)   | Action (1) | ... | Action (m-1) | Action (m) |
|             | 0       | 0            | 0          | 0   | 0            | 0          |
|             | 1       | 0            | 0          | 0   | 0            | 0          |
|             | 2       | 0            | 0          | 0   | 0            | 0          |
|             | ...     | 0            | 0          | 0   | 0            | 0          |
|             | n-1     | 0            | 0          | 0   | 0            | 0          |
|             | n       | 0            | 0          | 0   | 0            | 0          |

Empty Q-table. Image by [author](#).

### SARSA algorithm

SARSA is an **on-policy** algorithm, which is one of the areas differentiating it from Q-Learning (off-policy algorithm). **On-policy** means that during training, we use the same policy for the agent to **act** (acting policy) and to **update** the value function (updating policy). Meanwhile, with the **off-policy** approach, we use different policies for acting and updating. Now let's take a look at the SARSA algorithm itself:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Diagram illustrating the SARSA algorithm update equation:

- Updated Q estimate for state-action pair** (points to the left  $Q(S_t, A_t)$ )
- Learning Rate** ( $\alpha$ ) (points to the  $\alpha$  term)
- Reward** ( $R_{t+1}$ ) (points to the  $R_{t+1}$  term)
- Discount Factor** ( $\gamma$ ) (points to the  $\gamma$  term)
- Value of the next state-next action pair** ( $Q(S_{t+1}, A_{t+1})$ ) (points to the  $Q(S_{t+1}, A_{t+1})$  term)
- Current Q estimate for state-action pair** (points to the right  $Q(S_t, A_t)$ )

SARSA algorithm. Image by [author](#).

- Q is the **value function**, and the term on the left  $Q(S_t, A_t)$  is the **new value** for the specific state-action pair. Note, S refers to State, and A refers to Action.
- On the right-hand side of the equation, we find the same term  $Q(S_t, A_t)$ , which, in this case, is the **current value** for that same state-action pair.
- To update the current value, we take the **reward** ( $R_{t+1}$ ) following the action taken by the agent, add the **value for the next state-next action pair**  $\gamma Q(S_{t+1}, A_{t+1})$  **discounted by gamma**, and subtract the **current value**  $Q(S_t, A_t)$ .
- So, the terms in the square brackets produce a positive, zero or negative value, which leads to either increase, no change or a decrease in the new value of  $Q(S_t, A_t)$ . Note that we also apply a **learning rate (alpha)** to control the “size” of each update.

Since SARSA uses **Temporal Difference (TD) approach**, the algorithm will keep updating the Q-table after each step until we reach the maximum number of iterations or the solution converges to an optimal one.

## Comparison to Q-Learning

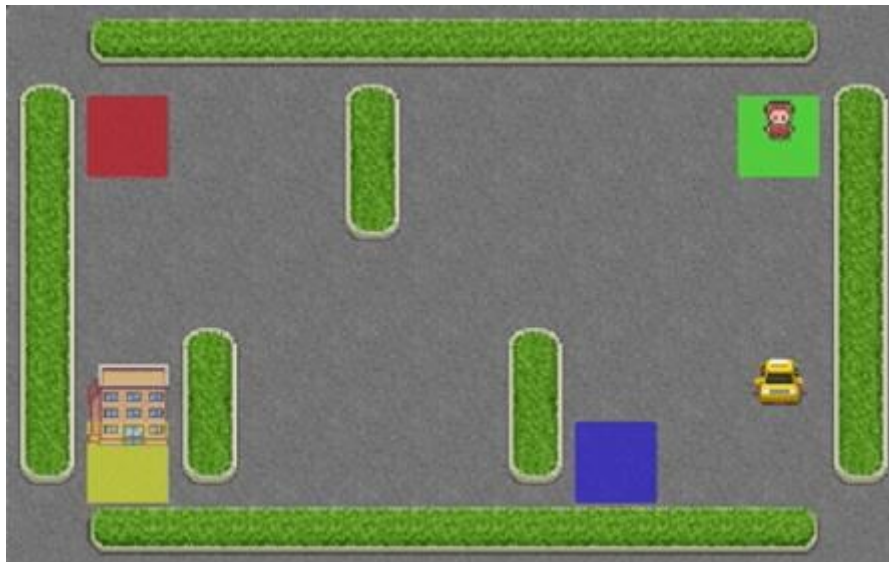
If we look at the equation used by the Q-Learning algorithm, we can see that the difference lies in how it selects the value of the next state. I.e., **Q-Learning takes the maximum value** for the next state based on the existing values in the Q-table. Meanwhile, **SARSA takes the value of the next state-next action pair**, as seen above.

$$\begin{array}{c}
 \text{Updated Q estimate for state-action pair} \\
 \downarrow \\
 Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \boxed{\max_a Q(S_{t+1}, a)} - Q(S_t, A_t)] \\
 \begin{array}{ccccccc}
 \uparrow & & \uparrow & & \uparrow & & \uparrow \\
 \text{Current Q estimate for state-action pair} & & \text{Reward received following the action taken} & & \text{Maximum value of the next state} & & \text{Current Q estimate for state-action pair}
 \end{array}
 \end{array}$$

Q-Learning algorithm. Image by [author](#).

## Python: Teaching an intelligent agent how to play a Taxi-v3 game

To better understand how it works, let's use the SARSA algorithm to teach an agent how to play a Taxi-v3 game available in [Open AI's Gym Environments](#). See the below gif image showing what the game looks like:



Gif image created by the [author](#) using the components from the [Taxi-v3 game](#).

The objective of the game is for the taxi to **pick up a passenger** (see a person) from a pick-up location, which can be either of the four colour squares and then **take the passenger to a drop-off location** (see a building), which also can be either of the four colour squares.

The default game rewards are as follows:

- -1 per step unless another reward is triggered
- +20 for delivering a passenger
- -10 for executing "pick-up" and "drop-off" actions illegally

### Setup

We will need to get the following libraries:

- [Open AI's Gym library](#) for the [Taxi-v3](#) game environment
- [Numpy](#) for data manipulation
- [Matplotlib](#) and built-in IPython library for displaying how the agent navigates its environment.

Let's import the libraries:

```
import numpy as np
print('numpy: %s' % np.__version__) # print version
```

```

# Note, need to 'pip install gym', and 'pip install gym[toy_text]'
# or use 'pip install gym\[toy_text\]' if zsh does not recongize the other command
import gymnasium as gym # for simulated environments
print('gym: %s' % gym.__version__) # print version
import matplotlib
import matplotlib.pyplot as plt # for displaying environment states
print('matplotlib: %s' % matplotlib.__version__) # print version
from IPython import display # for displaying environment states

```

The above code prints package versions used in this example:

```

numpy: 1.23.3
gym: 0.26.0
matplotlib: 3.6.0

```

Next, we set up a Taxi-v3 environment:

```

# Setup the environment
env = gym.make(id='Taxi-v3', # choose one of the existing environments
               max_episode_steps=None, # default=None, Maximum length of an episode
               (TimeLimit wrapper).
               autoreset=False, # default=None, Whether to automatically reset the
               environment after each episode (AutoResetWrapper).
               disable_env_checker=None, # default=None, If to run the env checker
               render_mode = 'rgb_array' # The set of supported modes varies per
               environment. (And some third-party environments may not support rendering at all.)
               )

```

After setting up the environment, let's check the **description**, **state space** and **action space**:

```

# Show environment description (map) as an array
print("Environment Array: ")
print(env.desc)

# Observation and action space
state_obs_space = env.observation_space # Returns sate(observation) space of the
environment.
action_space = env.action_space # Returns action space of the environment.
print("State(Observation) space:", state_obs_space)
print("Action space:", action_space)

```

```

Environment Array:
[[b'+ ' b'- ' b'- ' b'- ' b'- ' b'- ' b'- ' b'- ' b'- ' b'+']
 [b'| ' b'R' b':' b'' b'| ' b'' b':' b'' b':' b'G' b'|']
 [b'| ' b'' b':' b'' b'| ' b'' b':' b'' b':' b'' b'|']

```



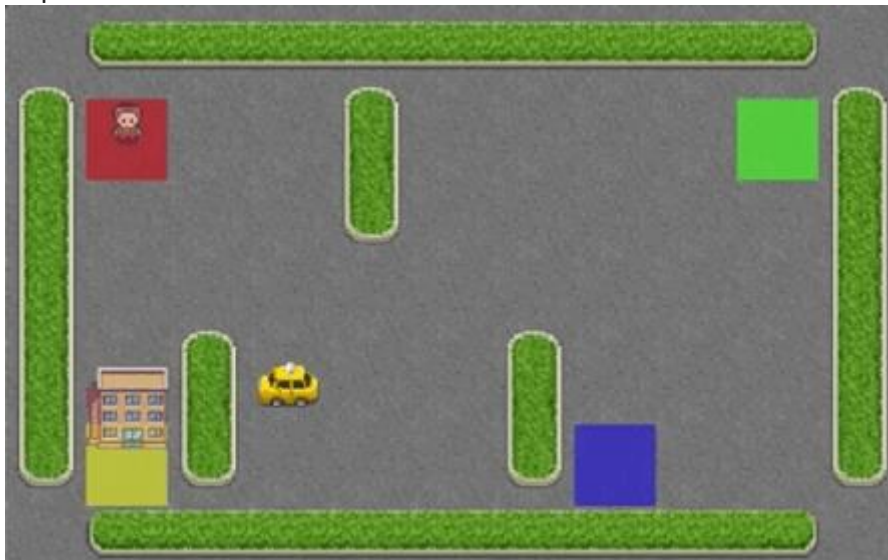
```

# Reset environment when done=True, i.e., when the agent falls into a Hole (H) or
reaches the Goal (G)
if done:
    # Render and display current state of the environment
    plt.imshow(env.render()) # render current state and pass to pyplot
    plt.axis('off')
    display.display(plt.gcf()) # get current figure and display
    display.clear_output(wait=True) # clear output before showing the next frame
    # Reset environment
    state, info = env.reset()

# Close environment
env.close()

```

Here is the output from the above code:



Gif image created by the [author](#) using the components from the [Taxi-v3 game](#).

## Model training

With the setup complete, let's use **SARSA** to find the best **policy( $\pi$ )** for our agent to follow in this game. We start by initialising a few parameters:

```

# SARSA parameters

alpha = 0.1 # learning rate

gamma = 0.95 # discount factor

# Training parameters

n_episodes = 100000 # number of episodes to use for training

n_max_steps = 100 # maximum number of steps per episode

```

```
# Exploration / Exploitation parameters

start_epsilon = 1.0 # start training by selecting purely random actions

min_epsilon = 0.0 # the lowest epsilon allowed to decay to

decay_rate = 0.00001 # epsilon will gradually decay so we do less exploring and more
exploiting as Q-function improves
```

To balance **exploration vs exploitation**, we will vary epsilon throughout training. We will start with epsilon=1 (pure exploration) and decay epsilon with every episode to gradually move from pure exploration to exploitation.

Next, let's initialise the Q-table. As we've seen in the previous section, it will be a 500 x 6 table with rows representing states and columns representing actions. We initialise the Q-table with all 0's since we do not know how valuable each state is before we start the training.

```
# Initial Q-table
# Our Q-table is a matrix of state(observation) space x action space, i.e., 500 x 6
Qtable = np.zeros((env.observation_space.n, env.action_space.n))

# Show
Qtable
```

Here is a snippet of initialised Q-table:

```
array([[0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0.]])
```

SARSA is an **on-policy** algorithm, so we will use the same **epsilon-greedy strategy** to act and update the Q-table. Let's now define functions to use in training and evaluation:

```
# This is our acting policy (epsilon-greedy), which selects an action for
exploration/exploitation during training
def epsilon_greedy(Qtable, state, epsilon):
    # Generate a random number and compare to epsilon, if lower then explore, otherwise
    exploit
    randnum = np.random.uniform(0, 1)
    if randnum < epsilon:
        action = env.action_space.sample() # explore
    else:
        action = np.argmax(Qtable[state, :]) # exploit
```

```

    return action

# This function is to update the Qtable.
# It is also based on epsilon-greedy approach because the next_action is decided by
epsilon-greedy policy
def update_Q(Qtable, state, action, reward, next_state, next_action):
    #  $Q(S_t, A_t) = Q(S_t, A_t) + \alpha [R_t + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$ 
    Qtable[state][action] = Qtable[state][action] + alpha * (reward + gamma *
(Qtable[next_state][next_action]) - Qtable[state][action])
    return Qtable

# This function (greedy) will return the action from Qtable when we do evaluation
def eval_greedy(Qtable, state):
    action = np.argmax(Qtable[state, :])
    return action

```

If you look closely at the **update\_Q** function, you will see that it contains the SARSA algorithm equation analysed in the previous section.

Finally, let's define our training function:

```

def train(n_episodes, n_max_steps, start_epsilon, min_epsilon, decay_rate, Qtable):
    for episode in range(n_episodes):

        # Reset the environment at the start of each episode
        state, info = env.reset()
        t = 0
        done = False

        # Calculate epsilon value based on decay rate
        epsilon = max(min_epsilon, (start_epsilon - min_epsilon)*np.exp(-decay_rate*episode))

        # Choose an action using previously defined epsilon-greedy policy
        action = epsilon_greedy(Qtable, state, epsilon)

        for t in range(n_max_steps):

            # Perform the action in the environment, get reward and next state
            next_state, reward, done, _, info = env.step(action)

            # Choose next action
            next_action = epsilon_greedy(Qtable, next_state, epsilon)

            # Update Q-table

```

```

Qtable = update_Q(Qtable, state, action, reward, next_state, next_action)

# Update current state
state = next_state
action = next_action

# Finish the episode when done=True, i.e., reached the goal or fallen into a hole

if done:
    break

# Return final Q-table
return Qtable

```

Now we can call the training function and see the results:

```

# Train
Qtable = train(n_episodes, n_max_steps, start_epsilon, min_epsilon, decay_rate, Qtable)

# Show Q-table
Qtable

```

Following the training, we get the Q-table as an output. However, it is difficult to say if it is optimal. Hence, we will need to do some additional evaluation. Here is a snippet of a Q-table post-training:

```

array([[ 0.      ,  0.      ,  0.      ,  0.      ,
         0.      ,  0.      ],
       [-15.88478151, -14.78900627, -16.69581259, -14.66434947,
        -12.6181495 , -23.80156577],
       [-6.19468628, -2.88294982, -6.49649702, -2.24076811,
        -0.99650814, -13.12010172],
       ...,
       [-4.25834456,  2.30997394, -5.05986368, -3.97161199,
        -12.53096203, -15.46710745],
       [-17.96569618, -12.49725207, -16.13686029, -10.22935569,
        -24.18094778, -27.10102464],
       [ 12.22333031,  6.43016417, 12.76697657, 15.36125068,
         4.18029065,  2.6316856 ]])

```

## Evaluation

Since it is difficult to judge the result by inspecting the Q-table, let's run 10,000 episodes to evaluate the agent's performance. We will use statistics such as **mean**, **standard deviation** and **min/max** of the rewards alongside the **distribution chart**.

```
def evaluate_agent(n_max_steps, n_eval_episodes, Qtable):
    # Initialize an empty list to store rewards for each episode
    episode_rewards=[]

    # Evaluate for each episode
    for episode in range(n_eval_episodes):

        # Reset the environment at the start of each episode
        state, info = env.reset()
        t = 0
        done = False
        tot_episode_reward = 0

        for t in range(n_max_steps):

            # Use greedy policy to evaluate
            action = eval_greedy(Qtable, state)

            # Pass action into step function
            next_state, reward, done, _, info = env.step(action)

            # Sum episode rewards
            tot_episode_reward += reward

            # Update current state
            state = next_state

            # Finish the episode when done=True, i.e., reached the goal or fallen into a
hole
            if done:
                break

        episode_rewards.append(tot_episode_reward)

    mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)
```

```
return mean_reward, std_reward, episode_rewards
```

We call the above function and display the results using the following code:

```
# Evaluate

n_eval_episodes=10000
mean_reward, std_reward, episode_rewards = evaluate_agent(n_max_steps, n_eval_episodes,
Qtable)

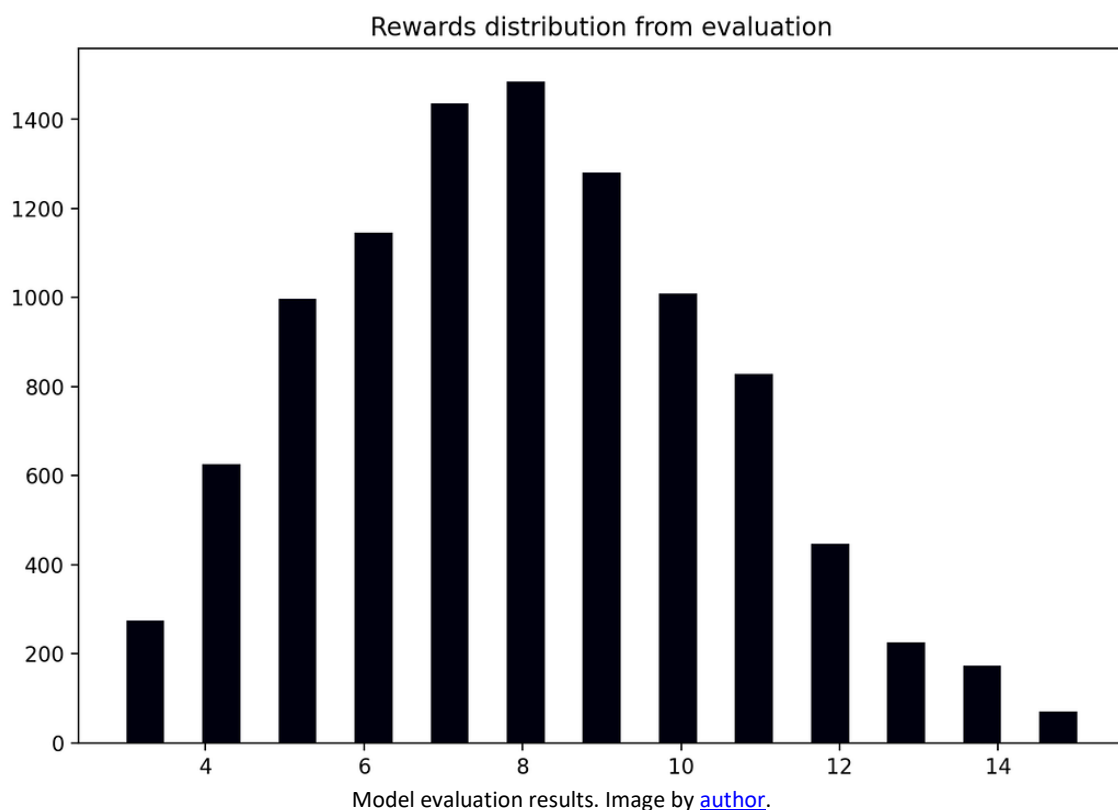
# Print evaluation results

print(f"Mean Reward = {mean_reward:.2f} +/- {std_reward:.2f}")
print(f"Min = {min(episode_rewards):.1f} and Max {max(episode_rewards):.1f}")

# Show the distribution of rewards obtained from evaluation

plt.figure(figsize=(9,6), dpi=200)
plt.title(label='Rewards distribution from evaluation', loc='center')
plt.hist(episode_rewards, bins=25, color='#00000f')
plt.show()

Mean Reward = 7.96 +/- 2.59
Min = 3.0 and Max 15.0
```



Let's unpack the results:

- **The average reward from 10,000 episodes is 7.96 +/- 2.59.** It indicates some variation in rewards, but that is because at the start of each episode, the locations

of a taxi, a passenger, and a drop-off change. Hence, each time we require a different number of steps to complete the task, so we don't always receive the same reward (each move is minus 1 point).

- **The minimum reward is 3, which confirms that we always get a positive reward.** Looking at the map, we can easily calculate that it takes 17 steps (16 moves + 1 pick-up action) if we initialised the taxi and passenger at opposite corners with a drop-off location being the same corner as the taxi's original position. Hence, choosing the perfect route for a most distant scenario would result in a reward of 3 ( $20 - 17$ ).
- **The maximum reward is 15, so let's see why we cannot expect any higher.** If a passenger is initialised at the drop-off location, the episode ends immediately (i.e., not a valid scenario, so there is no hope for scoring 20). Hence, the best case we can hope for is when the passenger and taxi are initialised in the same location. If we take the two closest colour squares (Red and Yellow), the agent will use 5 moves (1 to pick up and 4 to move) to transport the passenger from the pick-up location to the drop-off location, giving us the maximum possible reward of 15 ( $20 - 5$ ).
- **The reward distribution chart indicates a close-to-normal distribution of rewards.** So, while it does not prove that we have an optimal policy, it suggests that the agent is making rational moves.

One way to **ensure** that we have an optimal policy would be to calculate the minimum number of steps required for each initial state. Then validate that the agent never takes more than the minimum number of steps. However, I will leave this as an exercise for you. Instead, let me show a visualisation of an agent performing actions following our policy.

```
# Reset environment to initial state
state, info = env.reset()

# Cycle through 50 steps rednering and displaying environment state each time
for _ in range(50):

    # Render and display current state of the environment
    plt.imshow(env.render()) # render current state and pass to pyplot
    plt.axis('off')
    display.display(plt.gcf()) # get current figure and display
    display.clear_output(wait=True) # clear output before showing the next frame

# Use greedy policy to evaluate
```

```

action = eval_greedy(Qtable, state)

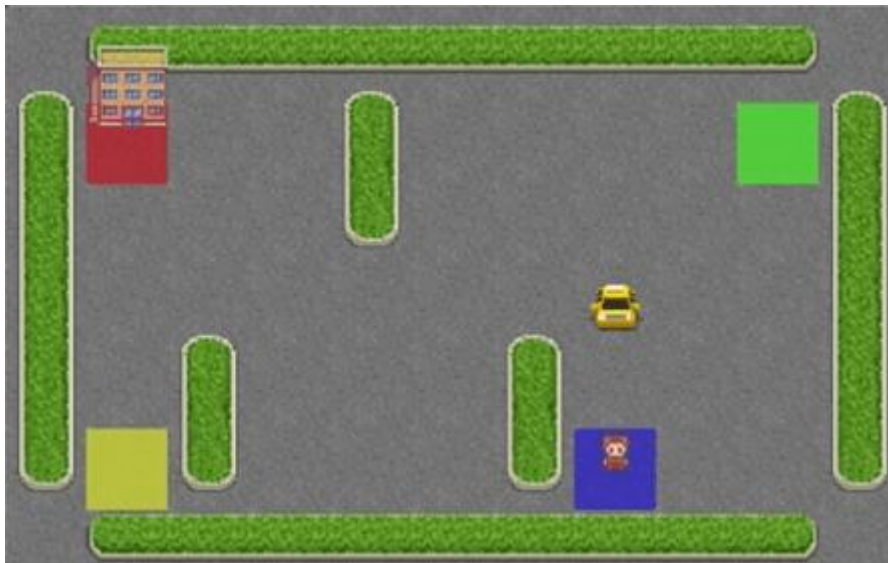
# Pass action into step function
state, reward, done, _, info = env.step(action)

# Reset environment when done=True, i.e. when the agent falls into a Hole (H) or
reaches the Goal (G)
if done:
    # Render and display final state of the environment
    plt.imshow(env.render()) # render current state and pass to pyplot
    plt.axis('off')
    display.display(plt.gcf()) # get current figure and display
    display.clear_output(wait=True) # clear output before showing the next frame
    state, info = env.reset()

env.close()

```

The above code gives us the following visualisation, which confirms that, at least in the two scenarios captured below, the agent has taken an optimal approach:



Gif image created by the [author](#) using the components from the [Taxi-v3 game](#).

## Final remarks

Now you know how SARSA works and how it differs from the standard Q-Learning algorithm. The Gym library provides many other environments (games), so I would encourage you to use your newly acquired knowledge to train an agent to play a different game. You can find complete Python code on this [GitHub repository](#). Note that there are two Jupyter Notebooks, one for the Taxi-v3 game and another for the Frozen-Lake game.